Agent2Agent Protocol (A2A)

Source: A2A Protocol Specification · Google Developers Blog · Google Cloud Blog

Version covered: v0.3 (July 2025), Linux Foundation project

Purpose: Everything you need to understand A2A before reading multi-agent papers and building AgentLink. Understand the MCP→A2A boundary clearly — this is the most common interview confusion point.

Contents

1 Why A2A Exists **2**

2 Core Concepts **2**
- 2.1 Agent Card . . . 2
- 2.2 Tasks . . . 3
- 2.3 Messages and Parts . . . 4
- 2.4 Artifacts . . . 4

3 Communication Protocol **4**
- 3.1 Transport . . . 4
- 3.2 Typical Interaction Flow . . . 4
- 3.3 Streaming . . . 5

4 Security Model **5**
- 4.1 Authentication . . . 5
- 4.2 Signed Agent Cards (v0.3) . . . 5
- 4.3 Capability Isolation . . . 6

5 MCP vs A2A: The Complete Picture **6**

6 A2A in the Context of Multi-Agent Papers **6**

7 Self-Check Questions **7**
- 7.1 Conceptual . . . 7
- 7.2 Protocol . . . 7
- 7.3 Architecture & Design . . . 7
- 7.4 Security . . . 7

8 Interview Questions **7**
- 8.1 Foundational . . . 7
- 8.2 Architecture . . . 8
- 8.3 Design Decisions . . . 8

Why A2A Exists

MCP solved the agent-to-tool problem. But as multi-agent systems emerged, a new problem appeared: **how do independent agents talk to each other?**

Each framework (LangGraph, CrewAI, AutoGen, ADK) built its own internal protocol. An agent built with LangGraph couldn't coordinate with a CrewAI agent without custom glue code. This was the **agent interoperability problem**.

The key distinction from MCP:

- MCP: agent ↔ tool (agent uses a hammer)
- A2A: agent ↔ agent (manager delegates to employee)

MCP defines how agents access tools and data. A2A defines how agents discover, communicate with, and delegate tasks to *other agents*, regardless of framework or vendor.

Google's motivation: After running large multi-agent enterprise systems, they saw repeating pain points: brittle one-off integrations, security gaps, no common handshake between agents from different vendors. A2A was open-sourced under Apache 2.0 and donated to the Linux Foundation in June 2025.

Ecosystem scale: As of v0.3, 150+ organizations support A2A — all major hyperscalers (Microsoft, AWS, Google Cloud), SAP, Salesforce, ServiceNow, Atlassian, PayPal, and major consulting firms.

Core Concepts

Agent Card

The **Agent Card** is A2A's fundamental discovery mechanism. Every A2A-compliant agent publishes a JSON document at a well-known URL.

Location: `/.well-known/agent-card.json`

Why `/.well-known/`? This follows RFC 8615 for well-known URIs — the same pattern used by OAuth servers, ACME certificate authorities, and OpenID Connect providers. It enables automatic discovery without prior configuration.

Listing 1: Agent Card Structure

```
{
  "name": "Research Agent",
  "description": "Specializes in web research and information synthesis",
  "version": "1.2.0",
  "url": "https://research-agent.example.com",
  "capabilities": {
    "streaming": true,
    "pushNotifications": true
  },
  "skills": [
    {
      "id": "web-research",
      "name": "Web Research",
      "description": "Search and synthesize information from the web",
      "inputModes": ["text"],
      "outputModes": ["text", "data"]
    }
  ],
  "securitySchemes": {
```

```
    "google": {
      "openIdConnectSecurityScheme": {
        "openIdConnectUrl": "https://accounts.google.com/.well-known/openid-configuration"
      }
    }
  }
}
```

Agent Card contains: identity (name, description, version, URL), capabilities (streaming, push notifications), skills (what the agent can do, with input/output modalities), security schemes (how to authenticate), and optionally an extended card for authenticated clients.

Signed Agent Cards (v0.3): Agents can cryptographically sign their Agent Cards, providing verifiable proof that declared capabilities are authentic and haven't been tampered with.

Tasks

Tasks are the primary unit of work in A2A. When a client agent wants something done, it creates a Task and sends it to a remote agent.

Task lifecycle:

Listing 2: Task Lifecycle States

```
PENDING --> RUNNING --> COMPLETED
                   \--> FAILED
                   \--> CANCELED
                   \--> INPUT_REQUIRED  (agent needs more info from user)
```

Listing 3: Task Object Structure

```
{
  "id": "task-uuid-here",
  "contextId": "conversation-context-uuid",
  "status": {
    "state": "TASK_STATE_COMPLETED",
    "timestamp": "2025-04-17T17:47:09.680794Z"
  },
  "artifacts": [
    {
      "artifactId": "result-uuid",
      "name": "Research Summary",
      "parts": [{"text": "The research findings are..."}]
    }
  ]
}
```

Key design decisions:

- Tasks have a `contextId` — multiple tasks can share a conversation context, enabling multi-turn interactions
- Long-running tasks can be polled without holding the connection open
- Push notifications: if the client provides a callback URL, the server pushes status updates instead of requiring polling
- `INPUT_REQUIRED` state enables human-in-the-loop flows where the agent pauses and waits for user input

Messages and Parts

Messages in A2A are composed of **Parts** — typed content units that can be mixed within a single message.

Listing 4: Message with Mixed Parts

```
{
  "role": "agent",
  "parts": [
    {"text": "Here is the analysis:"},
    {
      "inlineData": {
        "mimeType": "image/png",
        "data": "base64encodedimage..."
      }
    },
    {
      "fileData": {
        "mimeType": "application/pdf",
        "fileUri": "gs://bucket/report.pdf"
      }
    }
  ]
}
```

Why Parts? Different agents support different content modalities (text, images, audio, video, structured data). Parts enable agents to negotiate what content types they can exchange — called **UX negotiation**. Agents can check what output modalities the receiving agent supports before sending.

Artifacts

Artifacts are the final outputs of a Task — the deliverable. Distinct from intermediate messages (conversational), Artifacts represent completed work products. They support Extensions for domain-specific metadata (e.g., citation information, compliance records).

Communication Protocol

Transport

A2A is built on standard web protocols:

- **HTTP/HTTPS** for request-response
- **JSON** for message serialization
- **JSON-RPC** for structured method calls
- **Server-Sent Events (SSE)** for streaming responses
- **gRPC** (added in v0.3) for lower-latency internal service communication

Why standard protocols? Enterprise firewalls are built for HTTP. Using HTTP means A2A works in any enterprise environment without special network configuration.

Typical Interaction Flow

Listing 5: Complete A2A Interaction Flow

```
1. Discovery
   Client --> GET /.well-known/agent-card.json --> Remote Agent
   Client reads capabilities, auth requirements, supported skills

2. Authentication
   Client --> OAuth flow --> Authorization Server --> [access token]

3. Task Creation
   Client --> POST /tasks (with message) --> Remote Agent
   Remote Agent --> {taskId, status: PENDING} --> Client

4. Execution (async)
   Remote Agent processes task...
   [push]: Remote Agent --> POST client_callback_url --> Client
   [poll]: Client --> GET /tasks/{id} --> {status: RUNNING}

5. Completion
   Remote Agent --> POST client_callback_url --> {status: COMPLETED, artifacts: [...]}
   OR
   Client --> GET /tasks/{id} --> {status: COMPLETED, artifacts: [...]}
```

Streaming

For tasks producing output incrementally, A2A supports streaming via SSE:

Listing 6: SSE Streaming Response

```
Client --> POST /tasks (streaming requested)
Server --> 200 OK, Content-Type: text/event-stream

data: {"type": "task_status", "status": "RUNNING"}
data: {"type": "artifact_chunk", "chunk": "The first finding..."}
data: {"type": "artifact_chunk", "chunk": "The second finding..."}
data: {"type": "task_status", "status": "COMPLETED"}
```

Security Model

Authentication

A2A uses **OAuth 2.1-compatible** authentication, the same standard as MCP's June 2025 update. The security scheme is declared in the Agent Card.

Supported schemes: OpenID Connect (Google, Microsoft identity), API Key, Bearer tokens, Custom schemes.

Workload Identity: For agent-to-agent communication within a cloud environment (e.g., GKE), agents can use workload identity — cryptographic proof of the agent's identity tied to its deployment infrastructure. This removes the need for manually managed API keys.

Signed Agent Cards (v0.3)

Agent Cards can be cryptographically signed, enabling:

- **Verification:** A client can verify the Agent Card was issued by a trusted authority
- **Tamper detection:** If the card is modified in transit, the signature breaks
- **Non-repudiation:** The agent cannot deny it published these capabilities

This directly addresses the supply chain attack where a malicious proxy modifies Agent Cards to manipulate routing.

Capability Isolation

A2A's **"opaque agent"** design principle: agents communicate via A2A without exposing their internal implementation, memory, or tools. One agent cannot directly access another agent's state — it can only send messages and receive artifacts.

This is critical for enterprise environments where agents may be operated by different teams with different trust levels.

MCP vs A2A: The Complete Picture

Dimension	MCP	A2A
Relationship	Agent ↔ Tool	Agent ↔ Agent
Metaphor	Worker uses a hammer	Manager delegates to employee
Discovery	`tools/list`	`/.well-known/agent-card.json`
Comm. unit	Tool call + result	Task with lifecycle
Execution	Synchronous (mostly)	Async with status tracking
State visibility	Tool stateless, server maintains	Agents opaque to each other
Authentication	OAuth 2.1 / API keys	OAuth 2.1 / OIDC / Workload ID
Streaming	SSE via Streamable HTTP	SSE for task output
Primary use	Extending agent capabilities	Multi-agent coordination

In practice, use both together. Official recommendation: use MCP for tools and data sources, use A2A for agent-to-agent coordination.

```
Orchestrator Agent
|-[A2A]-> Research Agent -> [MCP] -> Web Search Tool
\-[A2A]-> Analysis Agent -> [MCP] -> Code Execution Tool
```

A2A in the Context of Multi-Agent Papers

Paper Concept	A2A Equivalent
"Agent communication" in AutoGen	A2A message exchange
"Task delegation"	A2A Task creation + assignment
"Role-based agents" in MetaGPT	Agents with specific skills in Agent Card
"Agent discovery"	Agent Card at `/.well-known/`
"Hierarchical multi-agent"	Orchestrator using A2A to delegate
"Long-running task"	A2A Task with async status tracking
"Human-in-the-loop"	`INPUT_REQUIRED` task state

Specifically for AutoGen: AutoGen's `ConversableAgent` exchange maps to A2A message passing. A2A adds the enterprise layer: discovery, authentication, async tasks, and cross-framework interoperability.

Specifically for MetaGPT roles: Each MetaGPT role (PM, Developer, QA) would be a separate A2A agent with a specific set of skills in its Agent Card. The orchestrator discovers which agent has which skill and delegates accordingly.

Self-Check Questions

Conceptual

What problem does A2A solve that MCP doesn't? Give a concrete example.

What is an Agent Card and why is it located at `/.well-known/agent-card.json`?

Describe the complete Task lifecycle in A2A.

What is the `INPUT_REQUIRED` task state and when would you use it?

What is the "opaque agent" design principle and why is it important for security?

Protocol

What is the difference between a Message and an Artifact in A2A?

What are Parts and why does A2A use them instead of plain text?

How do push notifications work in A2A? What problem do they solve?

What did gRPC support add in v0.3 and when would you use it over HTTP?

Architecture & Design

Draw the interaction flow for an orchestrator agent delegating a research task via A2A.

When would you use MCP vs A2A? Give examples for each.

How would you design a multi-agent system where agents are built with LangGraph and CrewAI but need to coordinate?

What is workload identity and why is it preferable to API keys for agent-to-agent auth?

Security

What attack do Signed Agent Cards prevent?

How does A2A's capability isolation model protect enterprise deployments?

What is the supply chain risk for A2A and how would you mitigate it?

Interview Questions

Foundational

- "What is A2A and how is it different from MCP?" → Must nail the MCP=tool, A2A=agent boundary
- "Walk me through how two agents discover and communicate using A2A."
- "What is an Agent Card and what information does it contain?"

- "How does A2A handle long-running tasks?"

Architecture

- "You're designing a multi-agent system with research, analysis, and writing agents. How would you structure their communication using A2A and MCP?" → Expect to explain which layer handles what
- "How would you handle a scenario where an A2A task takes 2 hours to complete and the client might disconnect?" → Async task lifecycle, push notifications, poll-on-reconnect
- "How do you prevent an agent from impersonating another agent in A2A?" → Signed Agent Cards, OAuth authentication, workload identity

Design Decisions

- "Why does A2A use standard HTTP instead of a custom protocol?"
- "What is the role of `contextId` in A2A tasks and how does it enable multi-turn conversations?"
- "How would you implement graceful degradation when a remote A2A agent goes offline mid-task?"
- "What does it mean that A2A agents are 'opaque' to each other? Why is this a feature?"

Official Spec: https://a2a-protocol.org/latest/specification/
A2A GitHub: https://github.com/a2aproject/A2A
Google Announcement: https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/
v0.3 Update: https://cloud.google.com/blog/products/ai-machine-learning/agent2agent-protocol-is-gett